COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010 Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	07/10/2026	**DATE SUBMITTED:**	07/15/2026
NAME:	Realubit Seth Andrew I.	**INSTRUCTOR:**	Sir. Jimlord Quejado

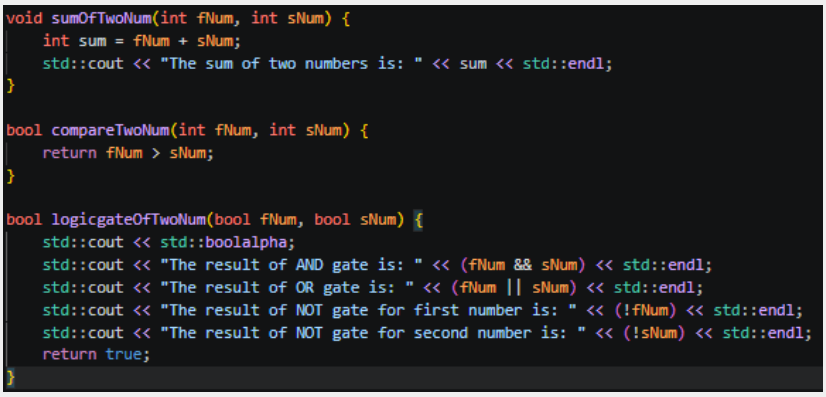
ACTIVITY NO. _1.1__ Hands-on Activity 1.1 Basic C++ Programming

1. PROCEDURE OUTPUT

Sections	Answer
Header File Declaration Section	`#include <iostream>`
Global Declaration Section	**N/A**

Class Declaration and Method Definition Section	`void sumOfTwoNum(int fNum, int sNum); // Sum of two numbers`
	`bool compareTwoNum(int fNum, int sNum); // Compare two numbers`
	`bool logicgateOfTwoNum(bool fNum, bool sNum); // Logic gate of two numbers`

Main Function	`int main() {`
	`  int a, b;`
	`  std::cout << "Enter two numbers: ";`
	`  std::cin >> a >> b;`
	`  sumOfTwoNum(a, b);`
	`  bool isGreater = compareTwoNum(a, b);`
	`  if (isGreater) {`
	`  std::cout << "The first number is greater than the second number." << std::endl;`
	`  } else {`
	`  std::cout << "The first number is not greater than the second number." << std::endl;`
	`  }`
	`  bool result = logicgateOfTwoNum(a > 0, b > 0);`
	`  return 0;`
	`}`

Method Definition	

```
void sumOfTwoNum(int fNum, int sNum) {
    int sum = fNum + sNum;
    std::cout << "The sum of two numbers is: " << sum << std::endl;
}

bool compareTwoNum(int fNum, int sNum) {
    return fNum > sNum;
}

bool logicgateOfTwoNum(bool fNum, bool sNum) {
    std::cout << std::boolalpha;
    std::cout << "The result of AND gate is: " << (fNum && sNum) << std::endl;
    std::cout << "The result of OR gate is: " << (fNum || sNum) << std::endl;
    std::cout << "The result of NOT gate for first number is: " << (!fNum) << std::endl;
    std::cout << "The result of NOT gate for second number is: " << (!sNum) << std::endl;
    return true;
}
```

Table 1-1. C++ Structure Code for Answer

ILO_A_REALUBIT.cpp | IILO_B_REALUBIT.cpp | ILO_C_REALUBIT.cpp

IILO_B_REALUBIT.cpp > ...

```cpp
#include <iostream>

class Triangle {
private:
    double totalAngle, angleA, angleB, angleC;
public:
    Triangle(double A, double B, double C);
    void setAngles(double A, double B, double C);
    const bool validateTriangle();
};
int main(){
    //driver code
    Triangle set1(40,30,100);
    if(set1.validateTriangle()){
        std::cout<<"The shape is a valid triangle.\n";
    } else {
        std::cout<<"The shape is not a valid triangle.\n";
    }

    set1.setAngles(60, 60, 60);
    if(set1.validateTriangle()){
        std::cout<<"The shape is a valid triangle.\n";
    } else {
        std::cout<<"The shape is not a valid triangle.\n";
    }
}
Triangle::Triangle(double A, double B, double C){
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = angleA + angleB + angleC;
}
void Triangle::setAngles(double A, double B, double C){
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = angleA + angleB + angleC;
}
const bool Triangle::validateTriangle(){
    return (totalAngle == 180);
}

```

FIgure 1, Code Snippet of ILO_B

Output Observation

When Triangle set1(40, 30, 110) is created:
*40 + 30 + 110 = 180; prints "The shape is a valid triangle."

When set1.setAngles(40, 30, 100) is called:
*40 + 30 + 100; prints "The shape is not a valid triangle."

Observation on <= vs ==

“return (totalAngle <= 180);” This is not entirely wrong since not all triangles add up to 180, but for consistency I made it to "return (totalAngle == 180);" **'const' in validate triangle** 'const' in the validate triangle is for the returned values so that it does not get touched after it is returned.

Table 1-2. ILO B output observations and comments

What procedures did you perform?

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO_C_REALUBIT.cpp > main()

```cpp
#include <iostream>
#include <cmath>

void swapVariables(int &a, int &b); // Swapping two numbers
void KelvintoFahrenheit(double kelvin); // Converting Kelvin to Fahrenheit
void DistanceBetweenTwoPoints(double x1, double y1, double x2, double y2); // Calculating distance between two points

int main(){
    int a = 5;
    int b = 10;
    std::cout<< "Before swap: a = " << a << ", b = " << b << std::endl;
    swapVariables(a, b);
    std::cout<< "After swap: a = " << a << ", b = " << b << std::endl;

    double kelvin;
    std::cout << "Enter temperature in Kelvin: ";
    std::cin >> kelvin;
    KelvintoFahrenheit(kelvin);

    double x1, y1, x2, y2;
    std::cout << "Enter coordinates of first point (x1 y1): ";
    std::cin >> x1 >> y1;
    std::cout << "Enter coordinates of second point (x2 y2): ";
    std::cin >> x2 >> y2;
    DistanceBetweenTwoPoints(x1, y1, x2, y2);

    return 0;
}

void swapVariables(int &a, int &b) {
    int temp = a;
    a = b;
    b = temp;
}

void KelvintoFahrenheit(double kelvin) {
    double fahrenheit = (kelvin - 273.15) * 9/5 + 32;
    std::cout << kelvin << " Kelvin is equal to " << fahrenheit << " Fahrenheit." << std::endl;
}

void DistanceBetweenTwoPoints(double x1, double y1, double x2, double y2) {
    double distance = std::sqrt(std::pow(x2 - x1, 2) + std::pow(y2 - y1, 2));
    std::cout << "The distance between the points (" << x1 << ", " << y1 << ") and ("<< x2 << ", " << y2 << ") is: " << distance << std::endl;
}
```

Figure 2, Code Snippet of Supplementary Activity Problem 1 to 3

While tackling the supplementary activity I was confused because I looked at the problem as a whole, I tried looking at the problem one-by-one. Focusing on the problem for swapping variables, I made a temporary variable then stored the value of a into b then finally swapped it with b again. For the distance between two points, I included 'cmath' to access functions like sqrt and power for convenience.

ILO_C_P4_REALUBIT.cpp > Triangle > validateTriangle()

```cpp
#include <iostream>
#include <cmath>

class Triangle {
private:
    double sideA, sideB, sideC;
    double totalAngle, angleA, angleB, angleC;
public:
    Triangle(double A, double B, double C, double sideA, double sideB, double sideC);
    void setSides(double sA, double sB, double sC);
    void setAngles(double A, double B, double C);
    void getArea(), getPerimeter(), getTriangleType();
    const bool validateTriangle();
};

int main(){
    //driver code
    Triangle set1(40,30,100,50,60,70);
    if(set1.validateTriangle()){
        std::cout<<"The shape is a valid triangle.\n";
    } else {
        std::cout<<"The shape is not a valid triangle.\n";
    }

    set1.setSides(50, 60, 70);
    if(set1.validateTriangle()){
        std::cout<<"The shape is a valid triangle.\n";
    } else {
        std::cout<<"The shape is not a valid triangle.\n";
    }

    set1.getArea();
    set1.getPerimeter();
    set1.getTriangleType();
}
```

Figure 3, Code Snippet Part 1 of Problem 4

```
38
39  Triangle::Triangle(double A, double B, double C, double sA, double sB, double sC){
40      sideA = sA;
41      sideB = sB;
42      sideC = sC;
43      angleA = A;
44      angleB = B;
45      angleC = C;
46      totalAngle = A + B + C;
47  }
48
49  void Triangle::setAngles(double A, double B, double C) {
50      angleA = A;
51      angleB = B;
52      angleC = C;
53      totalAngle = angleA + angleB + angleC;
54  }
55
56  void Triangle::setSides(double sA, double sB, double sC){
57      sideA = sA;
58      sideB = sB;
59      sideC = sC;
60      totalAngle = angleA + angleB + angleC;
61  }
62
63  const bool Triangle::validateTriangle(){
64      return (totalAngle == 180);
65  }
66
67  void Triangle::getArea() {
68      double s = (sideA + sideB + sideC) / 2;
69      double area = sqrt(s * (s - sideA) * (s - sideB) * (s - sideC));
70      std::cout << "The area of the triangle is: " << area << std::endl;
71  }
72
```

Figure 4, Code Snippet Part 2 of Problem 4

For lines 67 to 71, I used an entirely different formula for finding the area of a triangle called Heron's Formula found on the internet. While not exactly ideal since using the basic formula would prove to be simpler, I tried to challenge myself to add new variables inside the private and public class along with trying something new.

```
73  void Triangle::getPerimeter() {
74      double perimeter = sideA + sideB + sideC;
75      std::cout << "The perimeter of the triangle is: " << perimeter << std::endl;
76  }
77
78  void Triangle::getTriangleType() {
79      if (angleA < 90 && angleB < 90 && angleC < 90) {
80          std::cout << "The triangle is acute." << std::endl;
81      } else if (angleA > 90 || angleB > 90 || angleC > 90) {
82          std::cout << "The triangle is obtuse." << std::endl;
83      } else {
84          std::cout << "Others." << std::endl;
85      }
86  }
87
```

Figure 5, Code Snippet Part 3 of Problem 4

For the last part which is triangle type, I used a simple boolean for figuring out the type of triangle and followed the terms listed such as acute, obtuse, and others.

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Claude AI and Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of concepts, dividing the problems one-by-one, grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I asked the AI to divide the problems one-by-one and ask it to not give hints, for explanation of concepts I asked it to explain some forgotten concepts. For example including 'cmath' to allow the problem to be easily solved rather than using another syntax to solve the problem one at a time. Lastly, for grammarly, I simply used it for missed periods and commas while making the sentences.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.



Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.